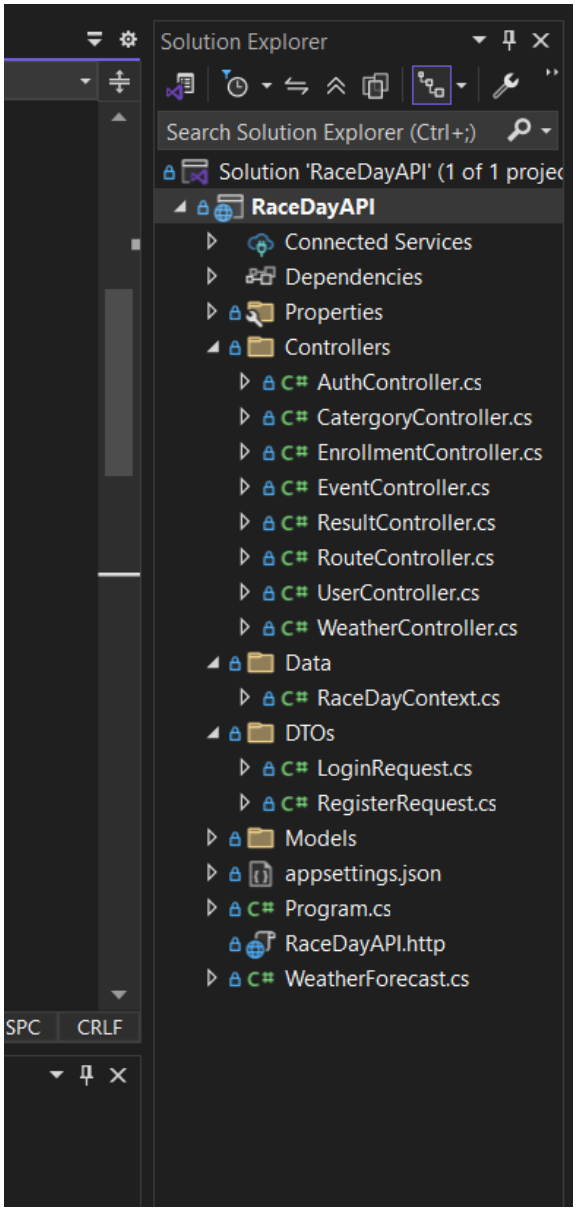
RaceDayAPI – Video Presentation Script

Introduction

Hi everyone. In this video, I will be explaining my RaceDayAPI project, including its code structure, main API endpoints and authentication system. RaceDayAPI is an event management API designed for running, walking and cycling events. It allows organisers to manage events and participants to register for events and access their information.

Code Structure

Starting with the code structure, the project is divided into different folders to keep everything organised.The Controllers folder handles requests sent to the API. For example, the EventsController handles requests related to events, while the AuthController handles registration and login.The Models folder contains the classes that represent the database tables. These include User, Event, Category, Enrolment, Result, Weather and Route. The Data folder contains RaceDayContext. This connects the application to the SQL Server database using Entity Framework Core. The DTOs folder contains RegisterRequest and LoginRequest. These classes define the information needed when a user registers or logs in.Program.cs configures the application services, database connection and authentication.Finally, appsettings.json contains application settings, including the database connection string.



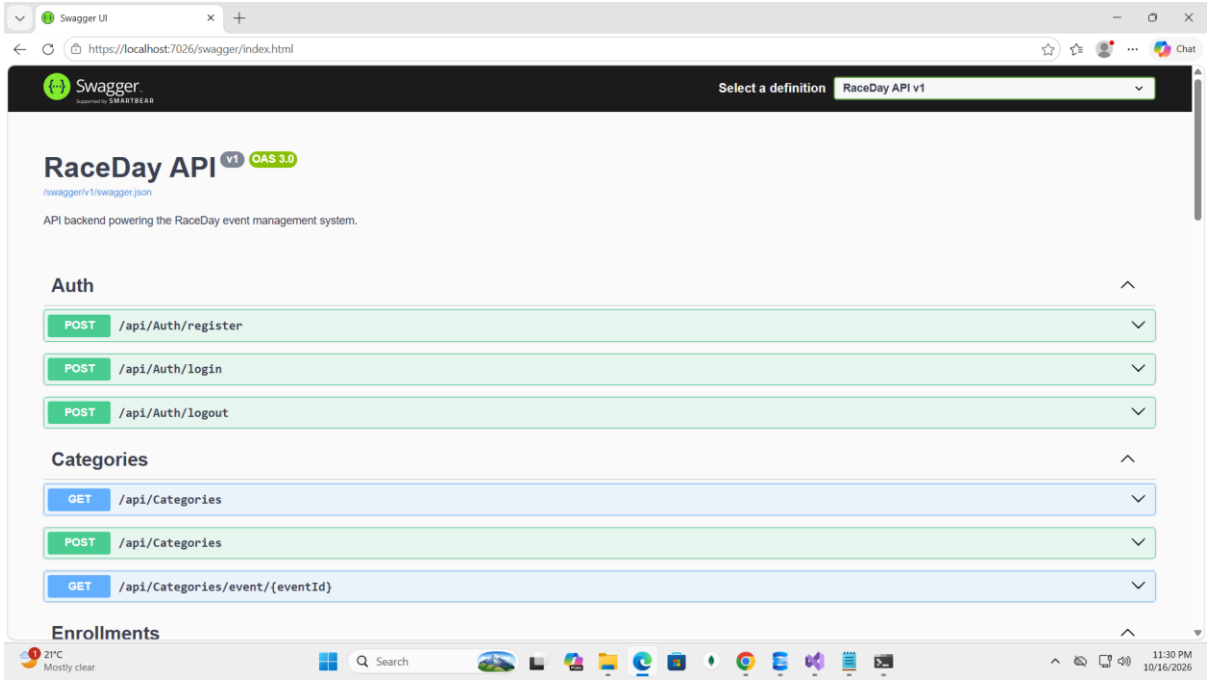
Database

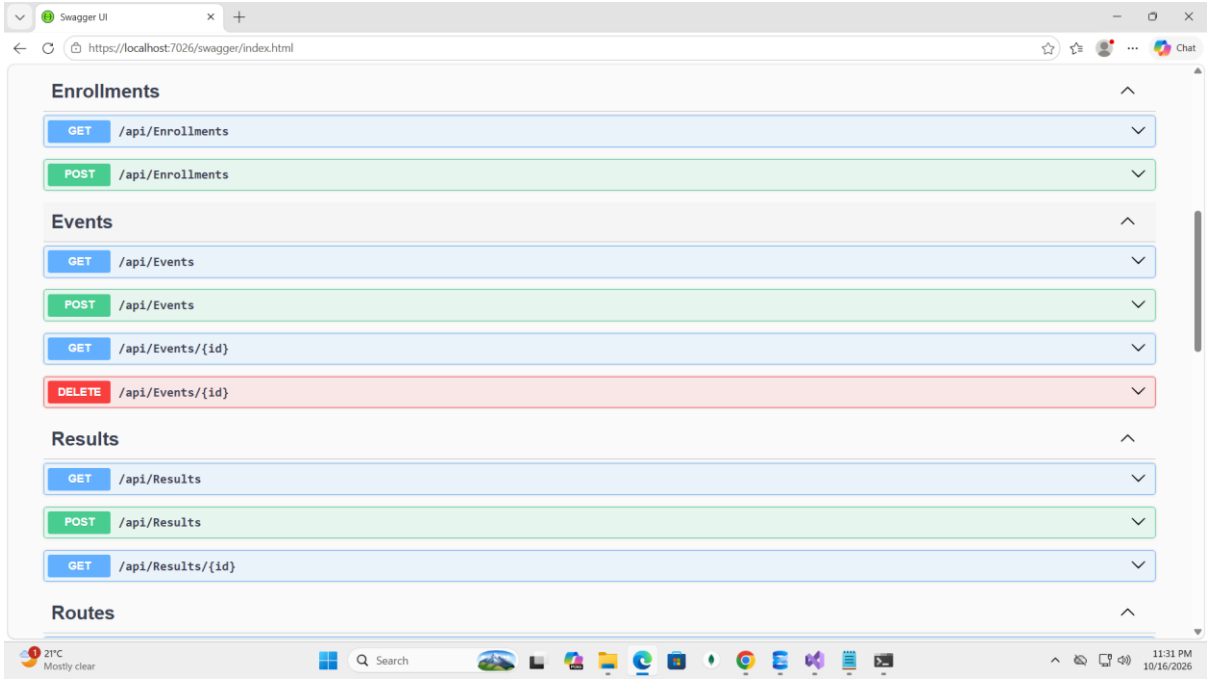
The project uses SQL Server with a database called RaceDayDB.The database stores user information, events, event categories, enrolments, results, weather information and routes.Entity Framework Core allows the C# application to communicate with the database without having to write a SQL query for every operation.

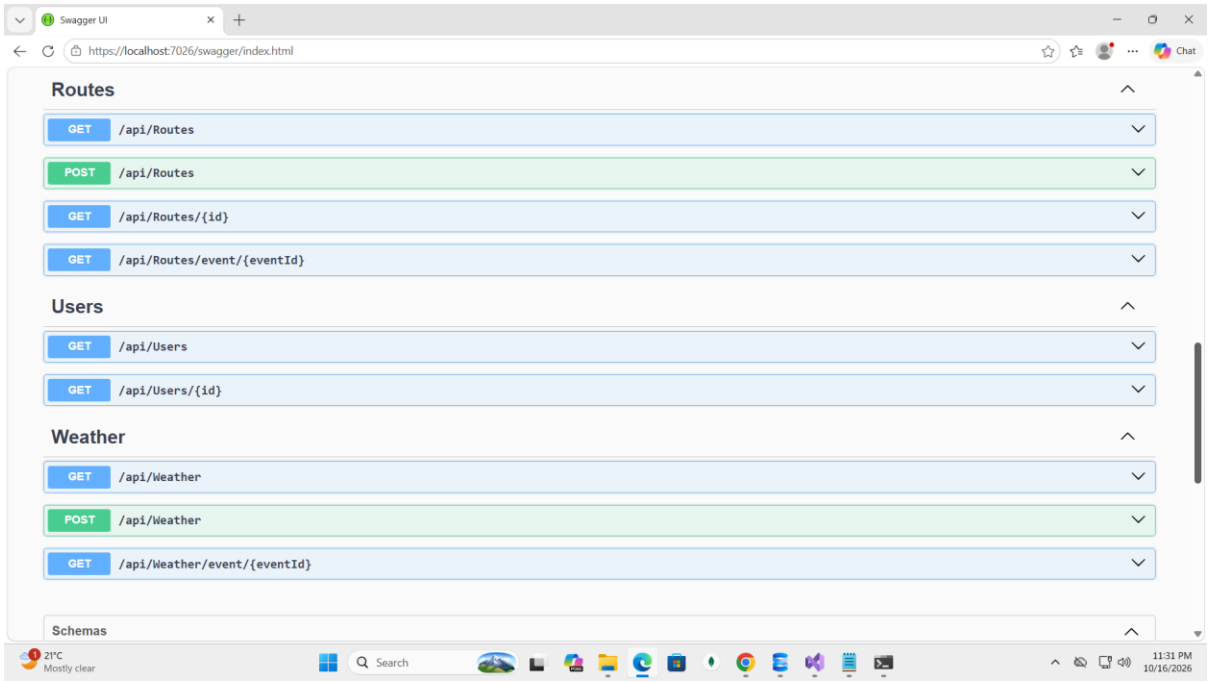
API Endpoints

Next, I will explain the main API endpoints. The AuthController handles registration, login and logout.The EventsController handles event-related requests, such as viewing events and managing event information.The CategoriesController manages the categories available for events.The EnrolmentsController handles participant enrolments. The ResultsController manages event results.The WeatherController provides weather information, while the RoutesController handles event route

information. The UsersController handles user-related information.The API uses HTTP methods such as GET to retrieve information, POST to create information, PUT to update information and DELETE to remove information.The available methods depend on the endpoint. I can use Swagger to view and test the endpoints.







Authentication

The project uses cookie-based authentication. When a user registers, the API receives their details through RegisterRequest. When they log in, the API checks their email address and password. After a successful login, the application creates an authenticated session using a cookie. The user's role is also used to determine which protected actions they can access. There are two user roles: Organiser and Participant. Organisers manage events, categories and results. Participants browse events, enter events and access their performance information. Role-based authorisation helps prevent users from accessing actions that are not permitted for their role.


RaceDayAPI
RaceDayAPI.Controllers.AuthC
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.EntityFrameworkCore;
using RaceDayAPI.Data;
using RaceDayAPI.Models;
using RaceDayAPI.DTOs;
using System.Security.Claims;
using System.Security.Cryptography;
namespace RaceDayAPI.Controllers
{
[ApiController]
[Route("api/[controller]")]
1 reference | Student, 32 minutes ago | 1 author, 1 change
public class AuthController : ControllerBase
{
private readonly RaceDayContext _context;
0 references | Student, 32 minutes ago | 1 author, 1 change
public AuthController(RaceDayContext context)
{
_context = context;
}
[HttpPost("register")]
0 references | Student, 32 minutes ago | 1 author, 1 change
public async Task<IActionResult> Register(RegisterRequest request)
{
if (request.Role != "Organiser" &&
request.Role != "Participant")
{
return BadRequest("Invalid role.");
}
if (await _context.Users.AnyAsync(
u => u.Email == request.Email))
{
return BadRequest("Email already exists.");
}
var user = new User
{
FirstName = request.FirstName,
LastName = request.LastName,
Email = request.Email,
PasswordHash = HashPassword(request.Password),
Role = request.Role,
CreatedAt = DateTime.Now
};
_context.Users.Add(user);
await _context.SaveChangesAsync();
return Ok("Registration successful.");


RaceDayAPI
RaceDayAPI.C
return Ok("Registration successful.");
}
[HttpPost("login")]
0 references | Student, 33 minutes ago | 1 author, 1 change
public async Task<IActionResult> Login(LoginRequest request)
{
var user = await _context.Users.FirstOrDefaultAsync(
u => u.Email == request.Email);
if (user == null ||
!VerifyPassword(request.Password, user.PasswordHash))
{
return Unauthorized("Invalid email or password.");
}
var claims = new List<Claim>
{
new Claim(
ClaimTypes.NameIdentifier,
user.UserID.ToString()),
new Claim(ClaimTypes.Name, user.Email),
new Claim(ClaimTypes.Role, user.Role)
};
var identity = new ClaimsIdentity(
claims,
CookieAuthenticationDefaults.AuthenticationScheme);
await HttpContext.SignInAsync(
CookieAuthenticationDefaults.AuthenticationScheme,
new ClaimsPrincipal(identity));
return Ok("Login successful.");
}
[HttpPost("logout")]
0 references | Student, 33 minutes ago | 1 author, 1 change
public async Task<IActionResult> Logout()
{
await HttpContext.SignOutAsync(
CookieAuthenticationDefaults.AuthenticationScheme);
return Ok("Logout successful.");
}
1 reference | Student, 33 minutes ago | 1 author, 1 change
private static string HashPassword(string password)
{
byte[] salt = RandomNumberGenerator.GetBytes(16);
byte[] hash = Rfc2898DeriveBytes.Pbkdf2(
70 %
No issues found



CI Using GitHub Actions

The project also uses GitHub Actions for continuous integration. When I push changes to GitHub, the workflow automatically restores the project dependencies and builds the application. A green tick in the Actions tab indicates that the build completed successfully. This helps identify build errors before making further changes.This workflow currently demonstrates continuous integration. It does not automatically deploy the application or run unit tests.

Conclusion

To conclude, RaceDayAPI uses ASP.NET Core Web API, C#, Entity Framework Core and SQL Server to provide an organised backend for event management.

The controllers handle requests, the models represent the database data, and authentication helps control access based on user roles.